

Technische Projektumsetzung

PdfFileMoverService: Aufbau, Komponenten und Verarbeitungsschritte

Stand: 18.08.2026

1. Technischer Überblick

Die Anwendung ist als .NET Worker Service umgesetzt und wird über den Generic Host gestartet. Durch die Verwendung von UseWindowsService kann die Anwendung als Windows-Dienst betrieben werden. Die Verarbeitung selbst findet in der Klasse Worker statt, die von BackgroundService erbt.

Host-Modell	Microsoft.Extensions.Hosting / Generic Host
Dienstintegration	UseWindowsService()
Hauptklasse	Worker : BackgroundService
Überwachung	FileSystemWatcher
Ereignis	Created
Verarbeitungsart	Asynchroner Task pro erkannter Datei
Logging	ILogger<Worker>

2. Programmstart

Der Einstiegspunkt erstellt den Standardhost, aktiviert die Windows-Service-Integration und registriert den Worker als Hosted Service. Danach wird der Host gebaut und ausgeführt.

Schritt	Technische Umsetzung
Host erstellen	Host.CreateDefaultBuilder(args)
Windows-Dienst aktivieren	UseWindowsService()
Worker registrieren	services.AddHostedService<Worker>()
Dienst starten	Build().Run()

3. Klasse Worker

Die Klasse Worker enthält die komplette Verarbeitungslogik. Sie verwaltet die Ordnerpfade, initialisiert den FileSystemWatcher, verarbeitet neue Dateien und gibt Ressourcen beim Stoppen des Dienstes wieder frei.

Element	Beschreibung
ILogger<Worker> _logger	Wird für die Protokollierung von Start, Verarbeitung

	und Fehlern verwendet.
FileSystemWatcher _watcher	Überwacht den Eingangsordner auf neue PDF-Dateien.
watchFolder	C:\rechnung, Ordner für neu eingehende PDF-Dateien.
targetFolder	C:\programm, Zielordner für verschobene Dateien.
processedFolder	C:\verarbeitet, Ablageordner für verarbeitete Kopien.

4. Initialisierung in StartAsync

1. Der Dienst schreibt eine Startmeldung in das Log.
2. Ein FileSystemWatcher wird für C:\rechnung und den Filter *.pdf erstellt.
3. Das Created-Ereignis wird mit der Methode OnCreated verbunden.
4. Unterordner werden nicht überwacht, weil IncludeSubdirectories auf false gesetzt ist.
5. Die Überwachung wird mit EnableRaisingEvents = true aktiviert.
6. Anschließend wird die Startlogik der Basisklasse aufgerufen.

5. Dateiverarbeitung in OnCreated

Wenn eine neue PDF-Datei erkannt wird, startet die Verarbeitung in einem separaten Task. Dadurch blockiert das Ereignis nicht direkt den FileSystemWatcher. Der Dateiname wird aus dem vollständigen Pfad ermittelt, danach werden Zielpfad und Verarbeitungspfad gebildet.

Phase	Aktion
Erkennung	Neue PDF-Datei wird über FileSystemEventArgs.FullPath gemeldet.
Pfadbildung	Dateiname, Zielpfad und Verarbeitungspfad werden mit Path.GetFileName und Path.Combine erstellt.
Bereitschaft	FileIsReady prüft, ob die Datei exklusiv geöffnet werden kann.
Verschieben	File.Move verschiebt die Datei vom Eingangsordner nach C:\programm.
Kopieren	File.Copy erstellt anschließend eine Kopie in C:\verarbeitet.
Fehlerbehandlung	Exceptions werden abgefangen und als Fehler

	protokolliert.
--	----------------

6. Bereitschaftsprüfung mit FileIsReady

Die Methode FileIsReady versucht, die Datei mit FileShare.None zu öffnen. Wenn dies gelingt, ist die Datei nicht durch einen anderen Prozess gesperrt und kann verarbeitet werden. Wenn das Öffnen fehlschlägt, wird false zurückgegeben und die Verarbeitung wartet erneut 500 Millisekunden.

Wichtig: Diese Prüfung verhindert, dass eine PDF-Datei bewegt wird, während sie noch geschrieben oder von einem anderen Prozess gesperrt ist.

7. StopAsync und Ressourcenfreigabe

Beim Stoppen des Dienstes wird der FileSystemWatcher über Dispose freigegeben. Danach wird eine Stoppmeldung geschrieben und die Stoplogik der Basisklasse aufgerufen.

8. ExecuteAsync

ExecuteAsync gibt Task.CompletedTask zurück. Die eigentliche Arbeit läuft nicht in einer Dauerschleife innerhalb von ExecuteAsync, sondern ereignisgesteuert über den FileSystemWatcher. Für diesen kleinen Dienst ist dieser Ansatz ausreichend, da die Verarbeitung durch Dateiereignisse ausgelöst wird.

9. Technische Hinweise für Betrieb

- Die Ordner C:\rechnung, C:\programm und C:\verarbeitet müssen auf dem Zielsystem vorhanden sein.
- Das Dienstkonto benötigt Lese- und Schreibrechte auf allen drei Ordnern.
- Bei Dateinamensgleichheit kann File.Move fehlschlagen, wenn im Zielordner bereits eine Datei mit gleichem Namen vorhanden ist.
- Logs sollten über die normale .NET-Logging-Konfiguration bzw. den Windows-Dienstkontext ausgewertet werden.
- Der Dienst verarbeitet aktuell nur Dateien, die nach dem Start neu erstellt werden.

10. Mögliche spätere Erweiterungen

Da das Projekt bewusst klein bleibt, sind Erweiterungen optional. Für einen stabileren Produktivbetrieb könnten später die Pfade in appsettings.json ausgelagert, Ordner beim Start geprüft bzw. angelegt, Dateikonflikte behandelt und ein kontrollierter Abbruch über CancellationToken ergänzt werden.

Kurzfazit: Die technische Umsetzung ist einfach, nachvollziehbar und für die aktuelle Aufgabe ausreichend. Eine große Refaktorisierung ist nicht erforderlich, sofern der Funktionsumfang gleich bleibt.